

Dataflow Analysis (2)

CSE552 Program Analysis — Lecture 8

Jaemin Hong

Live Variable Analysis

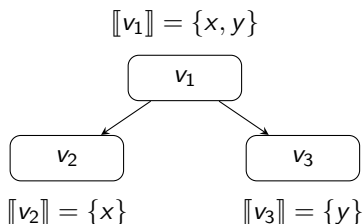
- ▶ A variable is *live* at a program point if there exists an execution where its value is read later in the execution without it being written to in between

```
// nothing is live
x = input();
// x is live
if input() {
    // x is live
    y = x;
    // y is live
} else {
    // nothing is live
    y = 1;
    // y is live
}
// y is live
z = y;
```

Live Variable Analysis — Motivation

- ▶ We can approximate the set of live variables using dataflow analysis
 - ▶ Application: register allocation
 - ▶ We want: the answer “not live” can be trusted and “live” is safe but useless

Live Variable Analysis — Abstract States



- ▶ State = $(\mathcal{P}(\text{Var}), \subseteq)$ — Power set lattice
- ▶ For each CFG node v , $\llbracket v \rrbracket$ denotes the set of variables live before the node
- ▶ $\text{JOIN}(v) = \bigsqcup_{u \in \text{succ}(v)} \llbracket u \rrbracket = \bigcup_{u \in \text{succ}(v)} \llbracket u \rrbracket$
 - ▶ This combines abstract states from the **successors**

Live Variable Analysis — Constraint Rule (Assignment)

► $x=e: \llbracket v \rrbracket = \text{JOIN}(v) \setminus \{x\} \cup \text{vars}(e)$

```
// y and z are live  
x = y + z;  
// x is live
```

Live Variable Analysis — Constraint Rules (Remaining)

- ▶ if x : $\llbracket v \rrbracket = \text{JOIN}(v) \cup \{x\}$
- ▶ entry: $\llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ return: $\llbracket v \rrbracket = \text{JOIN}(v) = \emptyset$

Available Expression Analysis

- ▶ A nontrivial expression (not a literal, not a variable) in a program is *available* at a program point if its current value has already been computed earlier in the execution

```
// nothing is available
x = y + 1;
// y + 1 is available
if input() {
    // y + 1 is available
    y = z + 1;
    // z + 1 is available
} else {
    // y + 1 is available
    x = z + 1;
    // y + 1 and z + 1 are available
}
// z + 1 is available
w = (z + 1) + (y + 1);
```

Available Expression Analysis — Motivation

- ▶ We can approximate the set of available expressions using dataflow analysis
 - ▶ Application: optimization (eliminating redundant computations)
 - ▶ We want: the answer “available” can be trusted and “not available” is safe but useless

Available Expression Analysis — Optimization Example

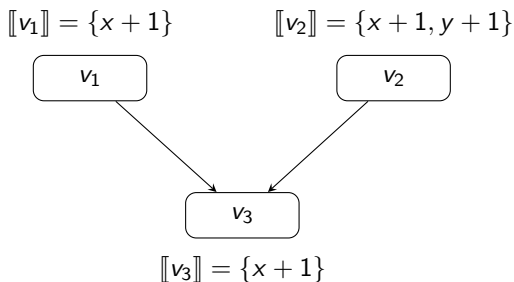
Before:

```
x = y + 1;
if input() {
    y = z + 1;
} else {
    x = z + 1;
}
w = (z + 1) + (y + 1);
```

After:

```
x = y + 1;
if input() {
    zplus1 = z + 1;
    y = zplus1;
} else {
    zplus1 = z + 1;
    x = zplus1;
}
w = zplus1 + (y + 1);
```

Available Expression Analysis — Abstract States



- ▶ State = $(\mathcal{P}(\text{Expr}), \supseteq)$ — **Reverse** power set lattice
- ▶ For each CFG node v , $\llbracket v \rrbracket$ denotes the set of expressions available after the node
- ▶ $\text{JOIN}(v) = \bigsqcup_{u \in \text{pred}(v)} \llbracket u \rrbracket = \bigcap_{u \in \text{pred}(v)} \llbracket u \rrbracket$
 - ▶ This combines abstract states from the **predecessors**

Available Expression Analysis — Constraint Rule (Assignment)

- ▶ $x=e: \llbracket v \rrbracket = (\text{JOIN}(v) \cup \text{exprs}(e)) \downarrow x$
 - ▶ $\downarrow x$ removes all expressions containing x
 - ▶ exprs collects all nontrivial expressions

$$\text{exprs}(x) = \emptyset$$

$$\text{exprs}(n) = \emptyset$$

$$\text{exprs}(\text{input}()) = \emptyset$$

$$\text{exprs}(e_1 \text{ op } e_2) = \{e_1 \text{ op } e_2\} \cup \text{exprs}(e_1) \cup \text{exprs}(e_2)$$

```
// x + 1 is available
x = x + (y + z);
// y + z is available
```

Available Expression Analysis — Constraint Rules (Remaining)

- ▶ **if** x : $\llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ **entry**: $\llbracket v \rrbracket = \top = \emptyset$
- ▶ **return**: $\llbracket v \rrbracket = \text{JOIN}(v)$

Very Busy Expression Analysis

- ▶ An expression is *very busy* if it will definitely be evaluated before its value changes

```
// nothing is very busy
x = input();
// x + 1 is very busy
if input() {
    // x + 1 is very busy
    y = x + 1;
} else {
    // x + 1 and y + 1 are very busy
    z = x + 1;
    // y + 1 is very busy
    w = y + 1;
}
```

Very Busy Expression Analysis — Motivation

- ▶ We can approximate the set of very busy expressions using dataflow analysis
 - ▶ Application: optimization (code hoisting)
 - ▶ We want: the answer “very busy” can be trusted and “not very busy” is safe but useless

Very Busy Expression Analysis — Optimization Example (if)

Before:

```
x = input();  
  
if input() {  
    y = x + 1;  
} else {  
    z = x + 1;  
    w = y + 1;  
}
```

After:

```
x = input();  
xplus1 = x + 1;  
if input() {  
    y = xplus1;  
} else {  
    z = xplus1;  
    w = y + 1;  
}
```

Very Busy Expression Analysis — Optimization Example (while)

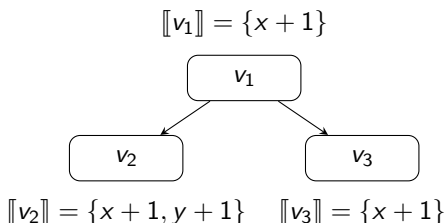
Before:

```
x = input();  
  
while input() {  
    y = x + 1;  
}  
z = x + 1;
```

After:

```
x = input();  
xplus1 = x + 1;  
while input() {  
    y = xplus1;  
}  
z = xplus1;
```


Very Busy Expression Analysis — Abstract States



- ▶ State = $(\mathcal{P}(\text{Expr}), \supseteq)$ — **Reverse** power set lattice
- ▶ For each CFG node v , $\llbracket v \rrbracket$ denotes the set of expressions very busy before the node
- ▶ $\text{JOIN}(v) = \bigsqcup_{u \in \text{succ}(v)} \llbracket u \rrbracket = \bigcap_{u \in \text{succ}(v)} \llbracket u \rrbracket$
 - ▶ This combines abstract states from the **successors**

Very Busy Expression Analysis — Constraint Rules

- ▶ $x=e: \llbracket v \rrbracket = (\text{JOIN}(v) \downarrow x) \cup \text{exprs}(e)$

```
// x + (y + z) and y + z are very busy
x = x + (y + z);
// x + 1 is very busy
```

- ▶ if $x: \llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ entry: $\llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ return: $\llbracket v \rrbracket = \top = \emptyset$

Reaching Definition Analysis

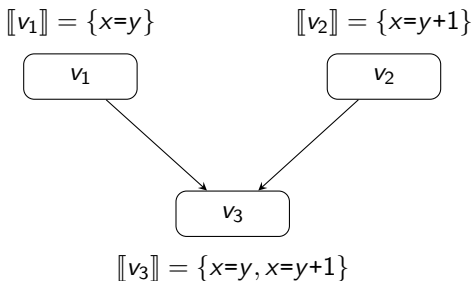
- *Reaching definitions* for a program point are those assignments that may have defined the current values of variables

```
if input() {  
    x = y;  
    // x = y is a reaching definition  
    x = y + 1;  
    // x = y + 1 is a reaching definition  
} else {  
    x = z + 1;  
    // x = z + 1 is a reaching definition  
}  
// x = y + 1 and x = z + 1 are reaching definitions  
return x;
```

Reaching Definition Analysis — Motivation

- ▶ We can approximate the set of reaching definitions using dataflow analysis
 - ▶ Application: def-use graph (useful for optimizations)
 - ▶ We want: the answer “not reaching” can be trusted and “reaching” is safe but useless

Reaching Definition Analysis — Abstract States



- ▶ State = $(\mathcal{P}(\text{Def}), \subseteq)$ — Power set lattice
 - ▶ Def $d ::= x=e$
- ▶ For each CFG node v , $\llbracket v \rrbracket$ denotes the set of definitions that may define values of variables at the program point after the node
- ▶ $\text{JOIN}(v) = \bigsqcup_{u \in \text{pred}(v)} \llbracket u \rrbracket = \bigcup_{u \in \text{pred}(v)} \llbracket u \rrbracket$
 - ▶ This combines abstract states from the **predecessors**

Reaching Definition Analysis — Constraint Rules

- ▶ $x=e: \llbracket v \rrbracket = (\text{JOIN}(v) \downarrow x) \cup \{x=e\}$
 - ▶ $\downarrow x$ removes all definitions of x

```
// x = y + 1 is a reaching definition
x = y;
// x = y is a reaching definition
```

- ▶ if $x: \llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ entry: $\llbracket v \rrbracket = \text{JOIN}(v) = \emptyset$
- ▶ return: $\llbracket v \rrbracket = \text{JOIN}(v)$

Time Complexity

- ▶ For SimpleWorkListAlgorithm, if $|\text{dep}(v)|$ is bounded by a constant for all nodes v , the worst-case time complexity is $O(n \cdot h \cdot k)$ where
 - ▶ n is the number of CFG nodes
 - ▶ h is the height of the lattice $L = \text{State}$
 - ▶ k is the worst-case time required to compute f_i
- ▶ $O(n \cdot m^2)$ where
 - ▶ n is the number of CFG nodes
 - ▶ m is the number of variables/expressions/definitions
 - ▶ Because $h = m$, $k = O(m)$

Forward vs Backward Analyses

- ▶ A *forward* analysis computes information about the past behavior
 - ▶ Examples: sign analysis, constant propagation analysis, available expression analysis, reaching definition analysis
 - ▶ The analysis starts at the entry node and propagates information forward in the CFG
 - ▶ JOIN is defined using pred
 - ▶ $dep = succ$
- ▶ A *backward* analysis computes information about the future behavior
 - ▶ Examples: live variables analysis and very busy expressions analysis
 - ▶ The analysis starts at the return node and propagates information backward in the CFG
 - ▶ JOIN is defined using succ
 - ▶ $dep = pred$

May vs Must Analyses

- ▶ A *may* analysis describes information that may possibly be true
 - ▶ Examples: live variables analysis and reaching definitions analysis
 - ▶ Typically uses a power set lattice
- ▶ A *must* analysis describes information that must definitely be true
 - ▶ Examples: available expression analysis and very busy expression analysis
 - ▶ Typically uses a reverse power set lattice

May vs Must Analyses — Soundness

- ▶ May $\neq$ Sound, Must $\neq$ Complete
- ▶ All these analyses are sound but not complete

May vs Must Analyses — Soundness (Live Variables)

- ▶ Live variables = $\{x\}$
 - ▶ Set of possible behavior: any execution that does not require any variable other than x to be live
 - ▶ Can have false positives (some such executions may be actually impossible)
 - ▶ Set of impossible behavior: any execution that requires some variable other than x to be live
 - ▶ No false negatives (such executions are indeed impossible)

May vs Must Analyses — Soundness (Available Expressions)

- ▶ Available expressions = $\{x + y\}$
 - ▶ Set of possible behavior: any execution that has already computed $x + y$
 - ▶ Can have false positives (some such executions may be actually impossible)
 - ▶ Set of impossible behavior: any execution that has not computed $x + y$
 - ▶ No false negatives (such executions are indeed impossible)

Classification of Dataflow Analyses

	Forward	Backward
May	Reaching definition analysis	Live variable analysis
Must	Available expression analysis	Very busy expression analysis

Example — Initialized Variable Analysis

```
if input() {  
    x = 1  
    // x is initialized  
    y = x + 1  
    // x and y are initialized  
} else {  
    y = 2  
    // y is initialized  
}  
// y is initialized  
z = y + x
```

Example — Initialized Variable Analysis (cont.)

- ▶ We want to know whether a certain variable is definitely initialized at a program point
 - ▶ Must analysis — $\text{State} = (\mathcal{P}(\text{Var}), \supseteq)$
- ▶ Initialization is a property of past
 - ▶ Forward analysis — $\text{JOIN}(v) = \bigsqcup_{u \in \text{pred}(v)} \llbracket u \rrbracket$
- ▶ $x=e$: $\llbracket v \rrbracket = \text{JOIN}(v) \cup \{x\}$
- ▶ $\text{if } x$: $\llbracket v \rrbracket = \text{JOIN}(v)$
- ▶ entry : $\llbracket v \rrbracket = \emptyset$
- ▶ return : $\llbracket v \rrbracket = \text{JOIN}(v)$

Transfer Functions

- ▶ All constraint functions are of the form $\llbracket v \rrbracket = t_v(\text{JOIN}(v))$
where $t_v : L \rightarrow L$
- ▶ Example: live variable analysis
 - ▶ $x=e$: $\llbracket v \rrbracket = \text{JOIN}(v) \setminus \{x\} \cup \text{vars}(e)$
 - ▶ **if** x : $\llbracket v \rrbracket = \text{JOIN}(v) \cup \{x\}$
 - ▶ **entry**: $\llbracket v \rrbracket = \text{JOIN}(v)$
 - ▶ **return**: $\llbracket v \rrbracket = \text{JOIN}(v)$

Transfer Functions (cont.)

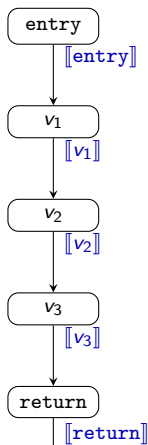
- ▶ t_v is called a *transfer function* for the CFG node
 - ▶ Forward analysis: input to the transfer function represents the state immediately before the node, and the output represents the state immediately after the node
 - ▶ Backward analysis: input to the transfer function represents the state immediately after the node, and the output represents the state immediately before the node
- ▶ Example: live variable analysis
 - ▶ $t_{x=e}(s) = s \setminus \{x\} \cup \text{vars}(e)$

Transfer Functions — Redundancy in SimpleWorkListAlgorithm

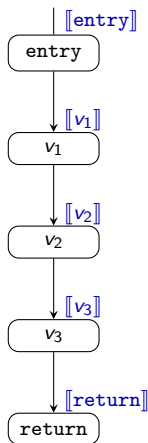
- ▶ In SimpleWorkListAlgorithm, $\text{JOIN}(v) = \sqcup \llbracket u \rrbracket$ is computed in each iteration
- ▶ However, $\llbracket u \rrbracket$ often has not changed since last iteration, so much of the computation is redundant
- ▶ We can use transfer functions to avoid redundancy
- ▶ Now, $x_i = \llbracket v_i \rrbracket$ is the state *before* v_i in forward analyses, and the state *after* v_i in backward analyses

Transfer Functions — State Positions (Forward)

W/o transfer functions

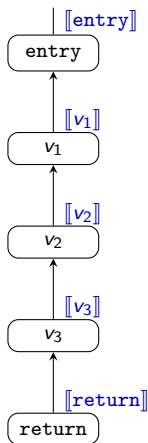


W/ transfer functions

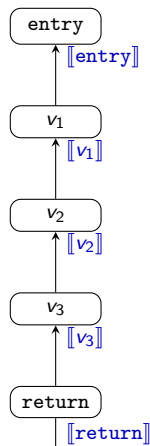


Transfer Functions — State Positions (Backward)

W/o transfer functions

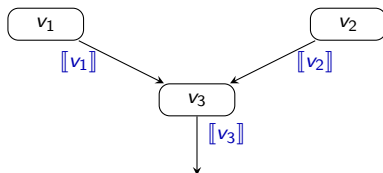


W/ transfer functions

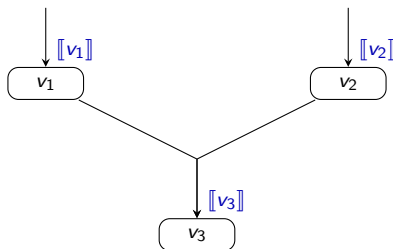


Transfer Functions — State Positions (Join)

W/o transfer functions:



W/ transfer functions:



PropagationWorkListAlgorithm

PropagationWorkListAlgorithm($t_1, \dots, t_n, s_{start}$):

$(x_1, x_2, \dots, x_n) \leftarrow (s_{start}, \perp, \dots, \perp)$

$W \leftarrow \{v_1, \dots, v_n\}$

while $W \neq \emptyset$ **do**

$v_i \leftarrow W.\text{removeOne}()$

$y \leftarrow t_{v_i}(x_i)$

for $v_j \in \text{dep}(v_i)$:

$z \leftarrow x_j \sqcup y$

if $x_j \neq z$:

$x_j \leftarrow z$

$W.\text{add}(v_j)$

return x

PropagationWorkListAlgorithm — Intuition

- ▶ This gives the same analysis results
- ▶ Intuition:
 - ▶ SimpleWorkListAlgorithm computes $x_3 = t_1(x_1) \sqcup t_2(x_2)$
 - ▶ PropagationWorkListAlgorithm computes $x_3 = x_3 \sqcup t_1(x_1)$ and $x_3 = x_3 \sqcup t_2(x_2)$, which is $x_3 = x_3 \sqcup t_1(x_1) \sqcup t_2(x_2)$
 - ▶ If f is monotone and $g(x) = f(x) \sqcup x$, then $\text{lfp}(g) = \text{lfp}(f)$

Summary

- ▶ Live variable analysis determines which variables may be needed in the future (backward, may analysis)
- ▶ Available expression analysis determines which expressions have already been computed (forward, must analysis)
- ▶ Very busy expression analysis determines which expressions will definitely be evaluated (backward, must analysis)
- ▶ Reaching definition analysis determines which assignments may define current variable values (forward, may analysis)
- ▶ Dataflow analyses are classified along two axes: forward/backward and may/must
- ▶ PropagationWorkListAlgorithm avoids redundant JOIN recomputation by propagating transfer function results incrementally